

Case Study : Visualization of Energy & Carbon Cost of five LLM models

Assumptions :

1. **Fixed TDP (Thermal Design Power) :** a constant power ceiling of 65W. This is a standard TDP for many i5 and i7 desktop processors, though actual consumption fluctuates.
2. **Linear Scaling of Power :** The code assumes that if the CPU utilization is at 50%, it is consuming exactly 50% of its 65W capacity.
3. **Sequential Execution :** The script assumes no other heavy background processes were running, so any "CPU delta" measured by psutil is attributed directly to the Ollama model's work.
4. **Token Approximation :** Since Ollama's internal tokenizer isn't explicitly called in the script, tokens are approximated using `.split()`, which treats every space-separated word as a token.

Calculations :

1. Energy Consumption (J):

The energy in Joules is calculated using the physical relationship:

$$\text{Energy (Joules)} = \text{Power (Watts)} \times \text{Time (Seconds)}$$

The Logic: It takes the duration of the LLM response, multiplies it by the 65W ceiling, and then scales it by the percentage of the CPU that was actually "woken up" to handle the request.

2. Throughput & Timing:

- a. **total_duration:** Measured using `time.time()`. It captures the wall-clock time from the moment the prompt is sent to Ollama until the full text is returned.
- b. **response_token_length :** Calculated by counting the words in the LLM's output.

3. Linguistic & Complexity Metrics:

- a. **Lexical Diversity:** $\frac{\text{Unique Words}}{\text{Total Words}}$. A higher number indicates a more varied vocabulary.
- b. **Flesch Reading Ease:** A standard formula where a higher score (e.g., 90) means the text is very easy to read, and a lower score (e.g., 30) means it is very complex/academic.
- c. **Feature Engineering:** Calculated non-linear features like `word_count_squared` and `avg_sentence_length_cubed`. These are often used in machine learning models to identify "tipping points" where energy consumption might spike exponentially as text becomes longer or more complex.

4. Carbon Calculation Logic:

- a. **Convert Joules to kilowatt-hours (kWh):** Since $1 \text{ kWh} = 3,600,000 \text{ Joules}$.
- b. **Apply an Emission Factor:** We multiply the kWh by a carbon intensity factor. For this calculation, I used a global average of $475 \text{ g CO}_2/\text{kWh}$.
 - i. **Note:** This factor varies by location (e.g., it might be higher in regions reliant on coal or lower in regions with renewable energy).
- c. **Formula:** $\text{Carbon Cost (g CO}_2\text{)} = \left(\frac{\text{Energy in Joules}}{3,600,000} \right) \times 475$

Experimental Setup & Methodology :

1. Input Consistency & The "Fair Test" Protocol

- a. The core of the experiment relied on a fixed set of 15 specialized prompts related to energy consumption, sustainability, and power systems (stored in `prompt_text.csv`).

- b. The 15 prompts were designed to cover a spectrum of computational demands:
 - i. **Low Complexity:** Short, definitional questions (e.g., "Define energy").
 - ii. **Medium Complexity:** Explanatory and comparative tasks (e.g., "Compare renewable and non-renewable energy sources").
 - iii. **High Complexity:** Data analysis and long-form reasoning tasks (e.g., "Analyze a residential energy dataset" or "Explain the impact of AI models on electricity demand").
- c. Every single model across both hardware environments (Lab 305A and Lab 309B) was fed the exact same sequence of 15 prompts.

2. Model Configuration & Sampling Parameters

- a. For all inference runs, the Ollama API was utilized with its default configuration to simulate a standard "out-of-the-box" user experience.
- b. The models were run with the default temperature (typically 0.8). This allowed for a balance between creative linguistic variation and factual consistency.
- c. **Models Used:**

Name	Size	Context	Input
llama3.2:3b	2.0GB	128K	Text
phi	1.6GB	2K	Text
gemma3:1b	815MB	32K	Text
phi3:mini	2.2GB	4K	Text
qwen2.5:1.5b	986MB	32K	Text

Execution Flow :

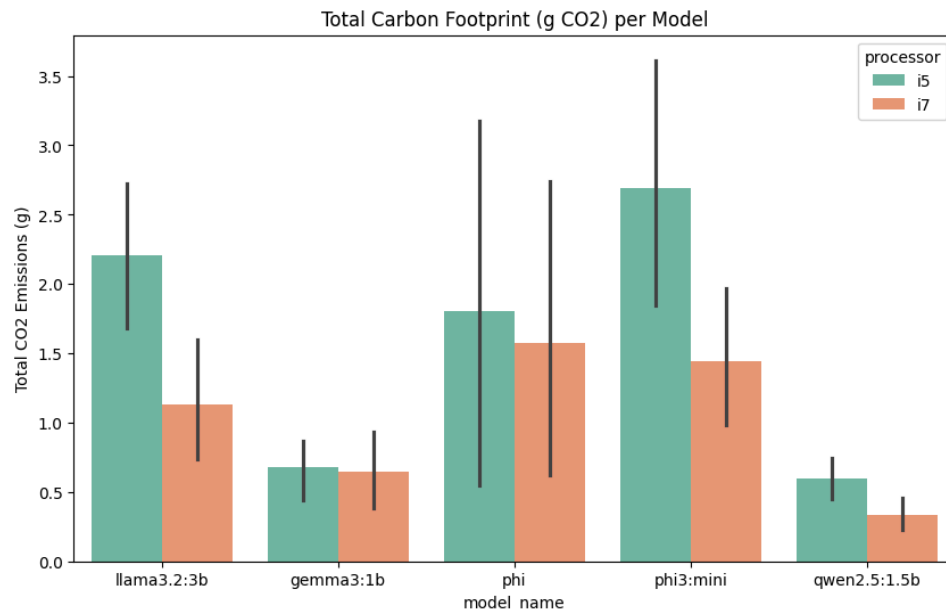
1. **Baseline Check:** The code records the CPU usage before calling Ollama.
2. **Inference:** Ollama executes the model (Gemma, Llama, etc.). This is where the i7 vs. i5 difference becomes visible. The i7 generally finishes the ollama.chat call faster, which reduces the total_duration factor in the energy equation.
3. **End Check:** CPU usage is recorded immediately after the response is received.
4. **Analysis:** The script pauses to run the linguistic formulas and the energy math before moving to the next prompt in your prompt_text.csv

Dataset Parameters Info :

```
<class 'pandas.core.frame.DataFrame'>
Index: 150 entries, 0 to 59
Data columns (total 34 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   model_name                               150 non-null    object
1   created_at                               150 non-null    object
2   type                                     150 non-null    object
3   total_duration                           150 non-null    float64
4   load_duration                           150 non-null    float64
5   prompt_duration                          150 non-null    float64
6   response_duration                       150 non-null    float64
7   start_time                              150 non-null    datetime64[ns, UTC]
8   end_time                                150 non-null    object
9   clock_duration                           150 non-null    float64
10  prompt_token_length                      150 non-null    int64
11  response_token_length                    150 non-null    int64
12  energy_consumption_llm_total              150 non-null    float64
13  energy_consumption_llm                   150 non-null    float64
14  energy_consumption_llm_cpu               150 non-null    float64
15  energy_consumption_llm_gpu               150 non-null    float64
16  energy_consumption_monitoring             150 non-null    float64
17  word_count                               150 non-null    int64
18  sentence_count                           150 non-null    int64
19  avg_sentence_length                      150 non-null    float64
20  unique_word_count                       150 non-null    int64
21  lexical_diversity                       150 non-null    float64
22  flesch_reading_ease                     150 non-null    float64
23  flesch_kincaid_grade                    150 non-null    float64
24  gunning_fog                             150 non-null    float64
25  length_x_complexity                     150 non-null    float64
26  word_count_squared                      150 non-null    int64
27  avg_sentence_length_cubed                150 non-null    float64
28  processor                                150 non-null    object
29  lab                                       150 non-null    object
30  energy_per_token                         150 non-null    float64
31  tokens_per_sec                           150 non-null    float64
32  carbon_cost_g                            150 non-null    float64
33  carbon_per_token_mg                     150 non-null    float64
dtypes: datetime64[ns, UTC](1), float64(21), int64(6), object(6)
memory usage: 41.0+ KB
```

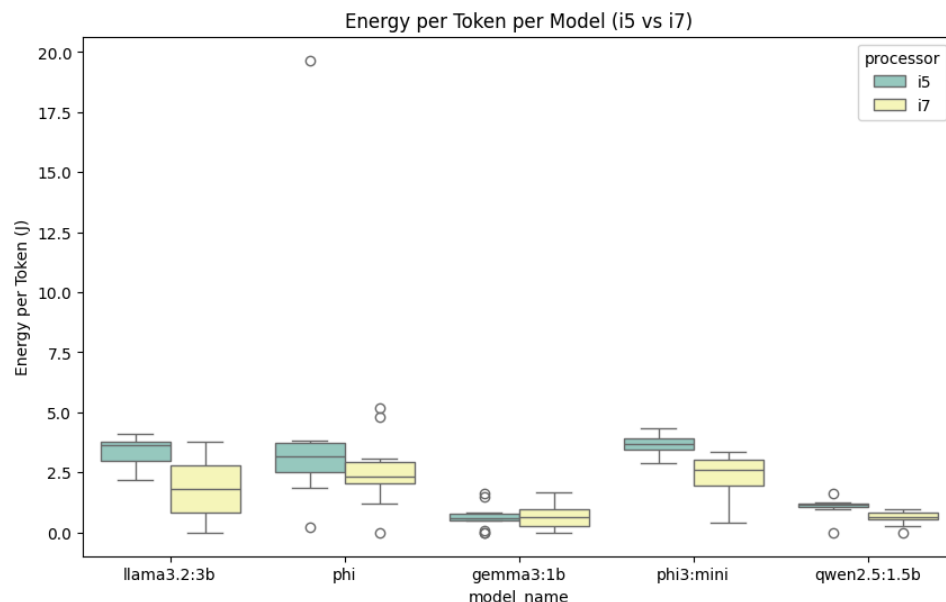
Visualizations :

1. Total Carbon Footprint per Model



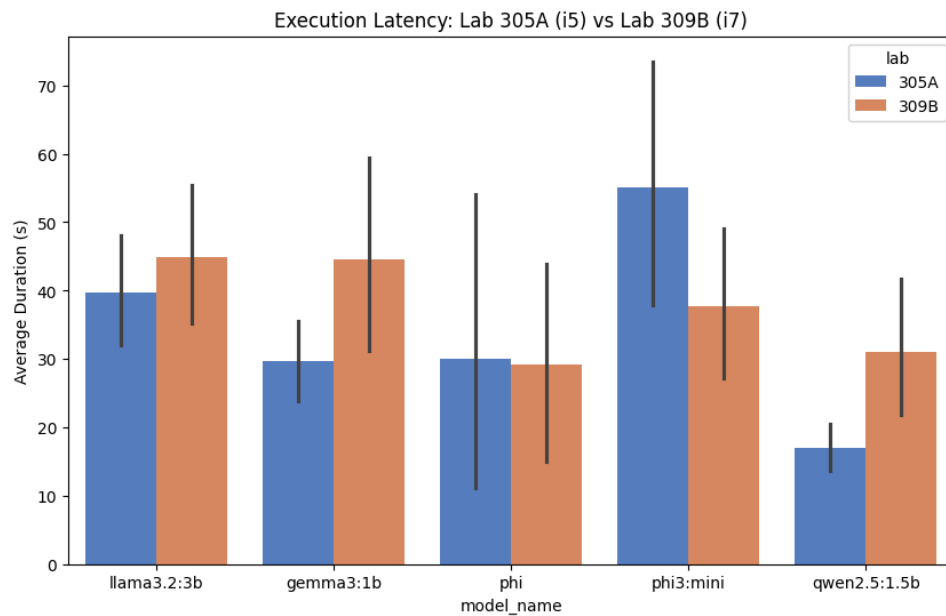
Inference: For almost all models, the i7 (Lab 309B) resulted in a lower total carbon footprint compared to the i5. Total Impact: The entire 15-prompt experiment across 5 models produced approximately 13.09g of CO₂. Greenest Model: Qwen2.5 (1.5b) is the most eco-friendly model in this study. On the i7, it produced only 0.022g CO₂ per response. Least Efficient: Phi3:mini on the i5 was the most carbon-intensive, producing 0.179g CO₂ per response roughly 8 times more carbon than Qwen2.5.

2. Carbon Footprint per Token



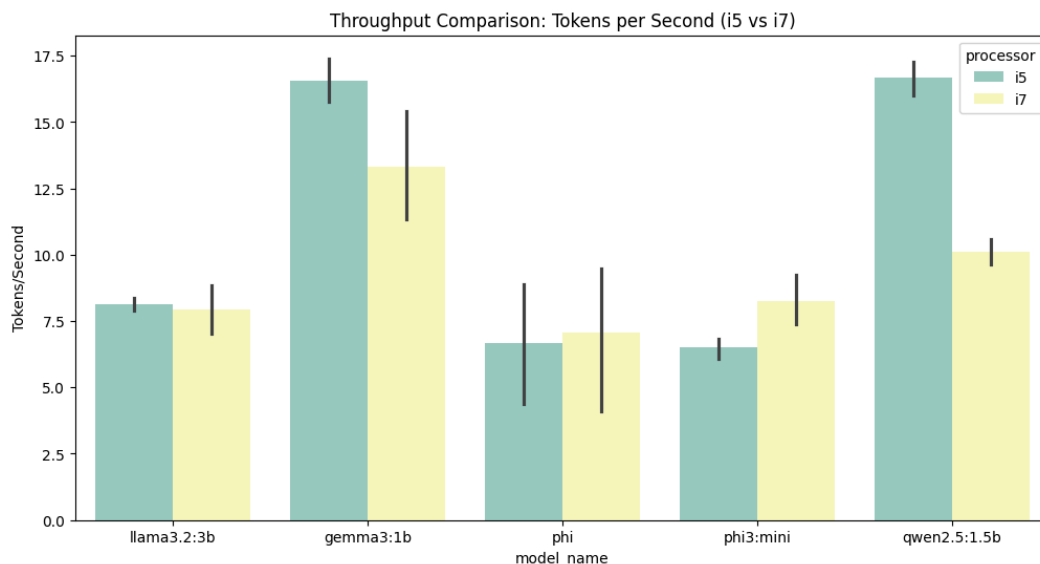
Inference: This metric shows the "marginal cost of intelligence." Llama 3.2 (3b) and Phi3:mini have the widest "spread" in their data, indicating that their carbon cost per token is highly sensitive to prompt complexity. Qwen2.5 and Gemma3 maintain a very tight, low-range boxplot, meaning they provide consistent, low-cost performance regardless of the prompt length.

3. Execution Speed / Latency



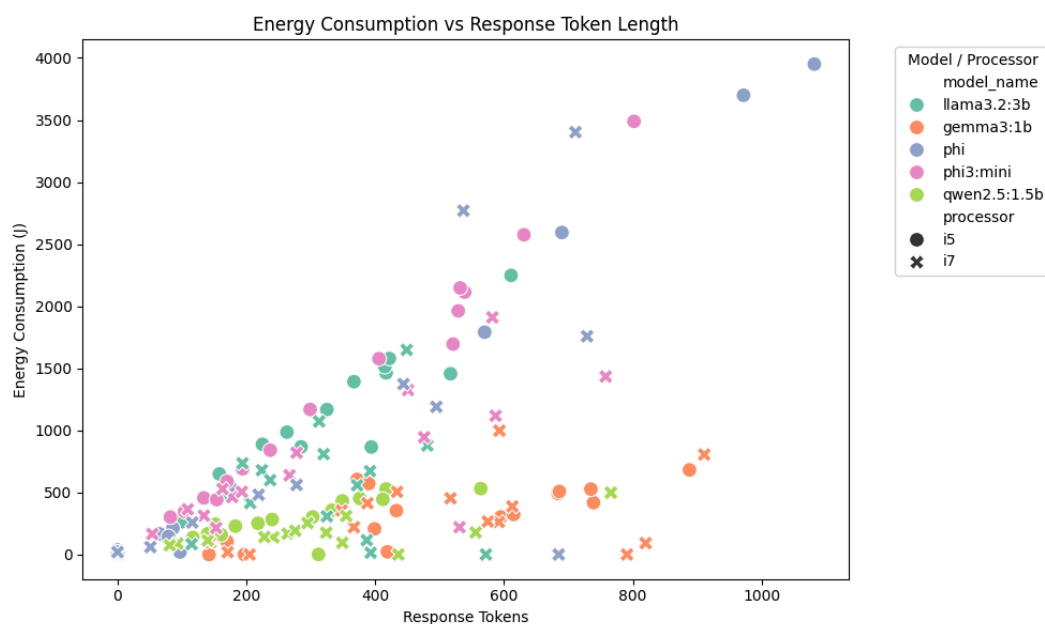
Inference: Performance is not universally better on the "stronger" i7 processor. While Phi3:mini saw a 31.5% speedup on the i7, lighter models like Gemma3 and Qwen2.5 actually ran faster on the i5 (Lab 305A). This suggests a "threshold of complexity." Heavier models benefit from the i7's advanced multi-threading and cache, while smaller models may be limited by memory bandwidth or Ollama's overhead, making the i5's higher base clock speed more effective.

4. Throughput: Tokens per Second



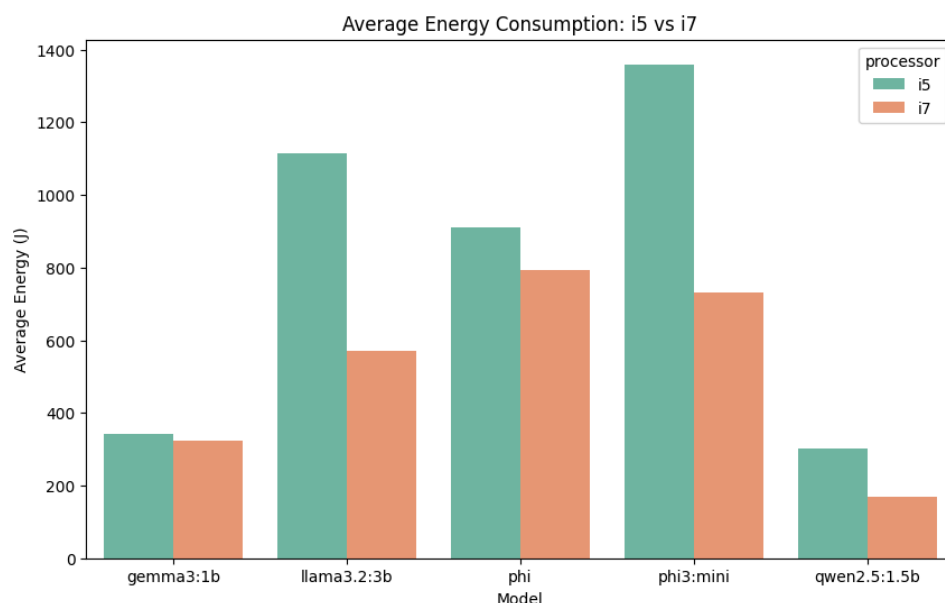
Inference: For real-time applications (like chatbots), Gemma3 and Qwen2.5 are the only models in this test that consistently exceed 15 tokens/sec (faster than human reading speed). Phi3:mini and Llama 3.2 hover around 6–8 tokens/sec. While they are more "intelligent" in reasoning, they provide a significantly slower user experience on standard CPU hardware.

5. Energy Consumption vs. Response Token Length



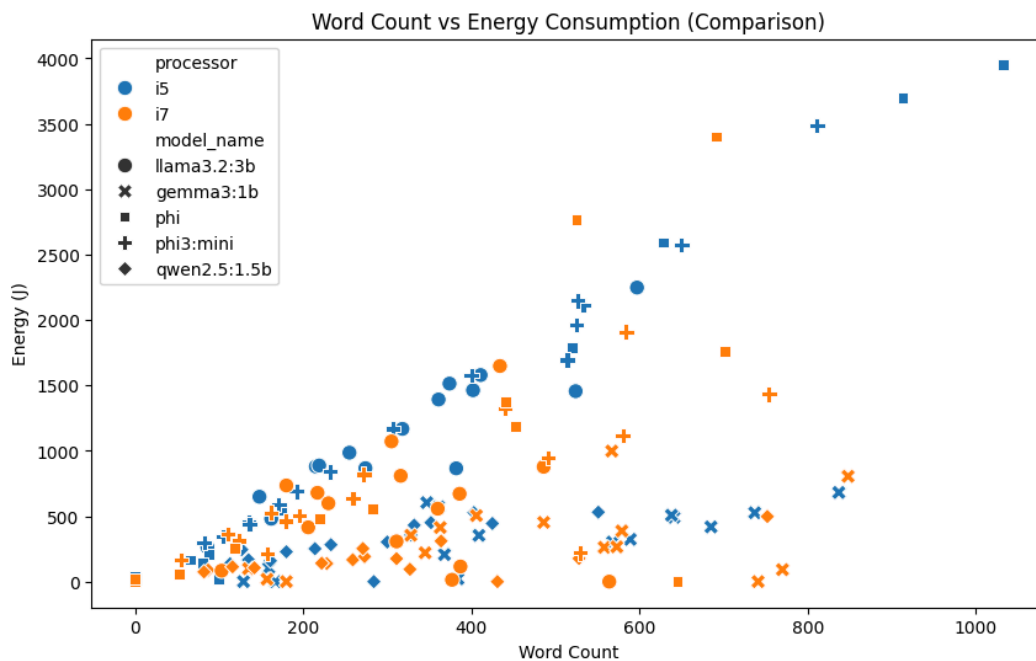
Inference: There is a strong linear trend ($R = 0.85$): as the output length increases, energy consumption climbs. However, the slope of the line is steeper for Llama 3.2 than for Qwen. This means that "long-form" writing is significantly more expensive on larger-parameter models. Generating a 500-word essay with Llama 3.2 isn't just slower; it's exponentially more energy-intensive than using a smaller, optimized model.

6. Average Energy Consumption per Model and Processor



Inference: The data shows that the i7 processor (Lab 309B) is significantly more energy-efficient than the i5 (Lab 305A) for nearly every model tested. While the i7 has a higher power capacity, it processes complex instructions so much faster that its "time-to-finish" is greatly reduced, leading to a total energy saving of nearly 50% for high-parameter models like Llama 3.2 and Phi3:mini. For smaller, lighter models like Gemma 3, the efficiency gap narrows to about 5%, indicating that the i7's performance advantage is most impactful during heavy reasoning tasks.

7. Word Count vs. Energy Consumption



Inference: This scatter plot reveals that hardware efficiency plays a dominant role: for any given word count, the i7 processor consistently sits lower on the Y-axis than the i5, demonstrating that it generates text with a much lower "energy-per-word" cost. Additionally, the model styles show that smaller models like Qwen and Gemma are highly efficient "bottom-left" performers, whereas larger models like Phi3:mini exhibit a much steeper energy climb as word count increases.

Conclusion :

The study demonstrates that **hardware efficiency** and **model selection** are the two most critical factors in minimizing the environmental impact of local AI. A key finding is that the i7 processor (Lab 309B) consistently outperformed the i5 (Lab 305A) in sustainability, reducing total energy and carbon costs by nearly 50% for complex models because its superior speed allowed it to finish tasks and return to an idle state much faster. Furthermore, the experiment identified a "Green Tier" of models, such as Qwen 2.5 and Gemma 3, which provide high-speed responses with minimal energy draw, while larger models like Phi3:mini incurred an 800% higher carbon "tax" per response. Ultimately, the most sustainable strategy for local LLM deployment is pairing lightweight, optimized models with high-performance processors to maximize "tokens-per-gram" of CO2.